NVIDIA

Scheduling recipes

Andrea Righi
<arighi@nvidia.com>

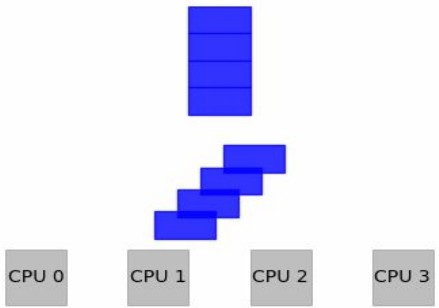


Kernel Recipes

What is a scheduler?

- Kernel component that determines
  - **Where** each task needs to run
  - **When** each task needs to run
  - For **how long** each task needs to run

- Scheduler
  - CPU allocator (**space**)
  - Task allocator (**time**)



Kernel Recipes

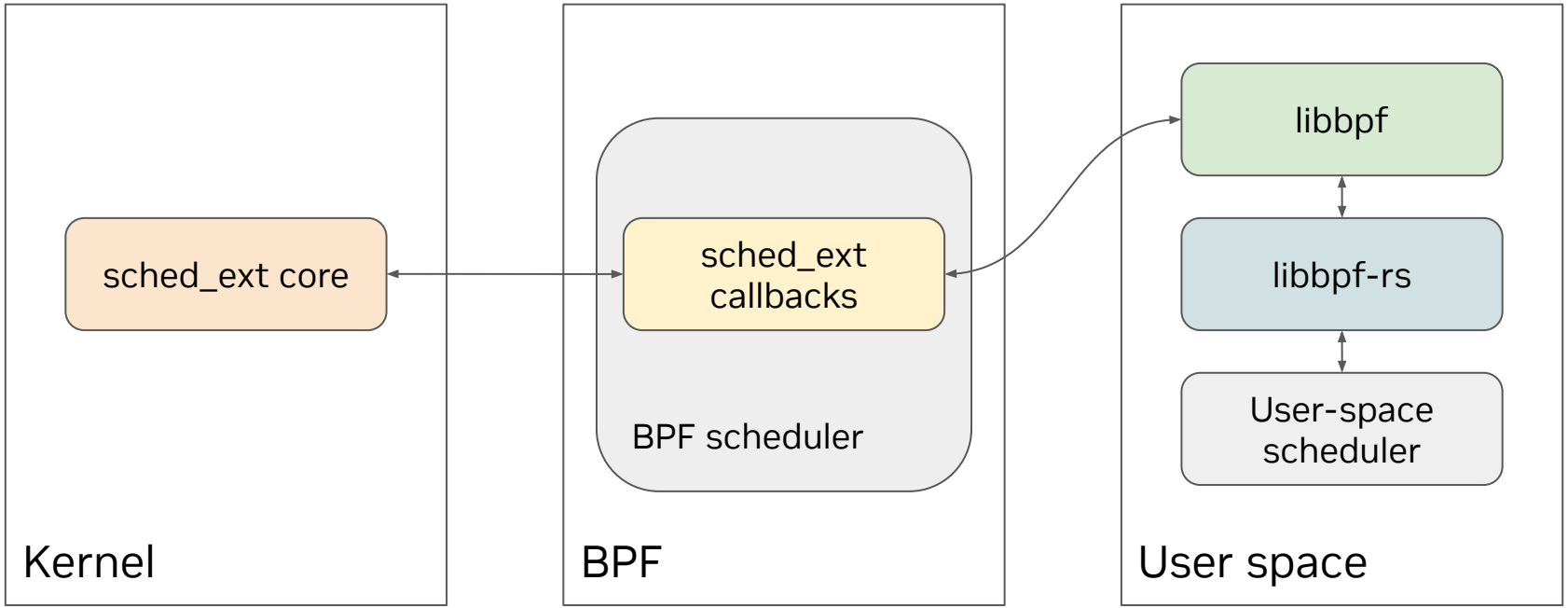
sched_ext: extensible scheduler class

- Technology in the Linux kernel that allows to implement scheduling policies as BPF programs (GPLv2)
- Available since Linux v6.12
- Key features
  - Bespoke scheduling policies
  - Rapid experimentation
  - Safety (can’t crash the kernel)



Kernel Recipes

BPF + user-space schedulers



Kernel Recipes

Gaming: fps consistency



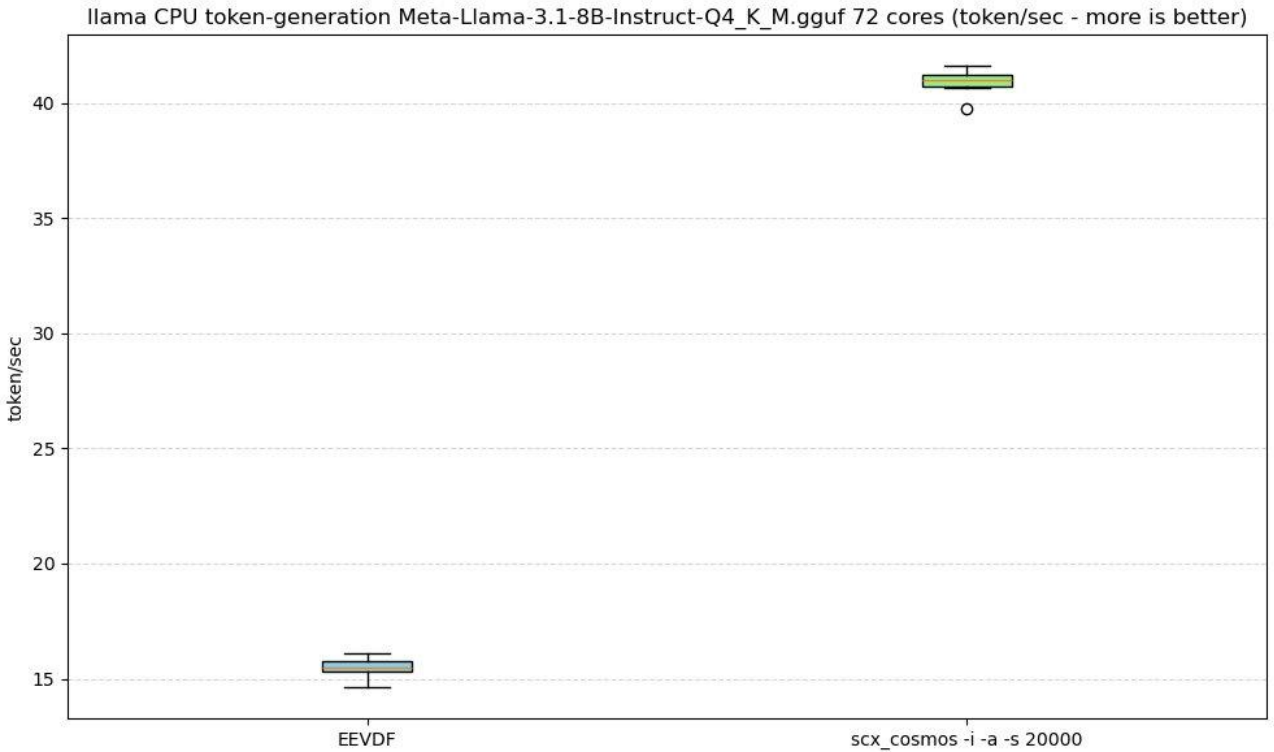
EEVDF



scx_flash

Kernel Recipes

Server workload: Llama benchmark



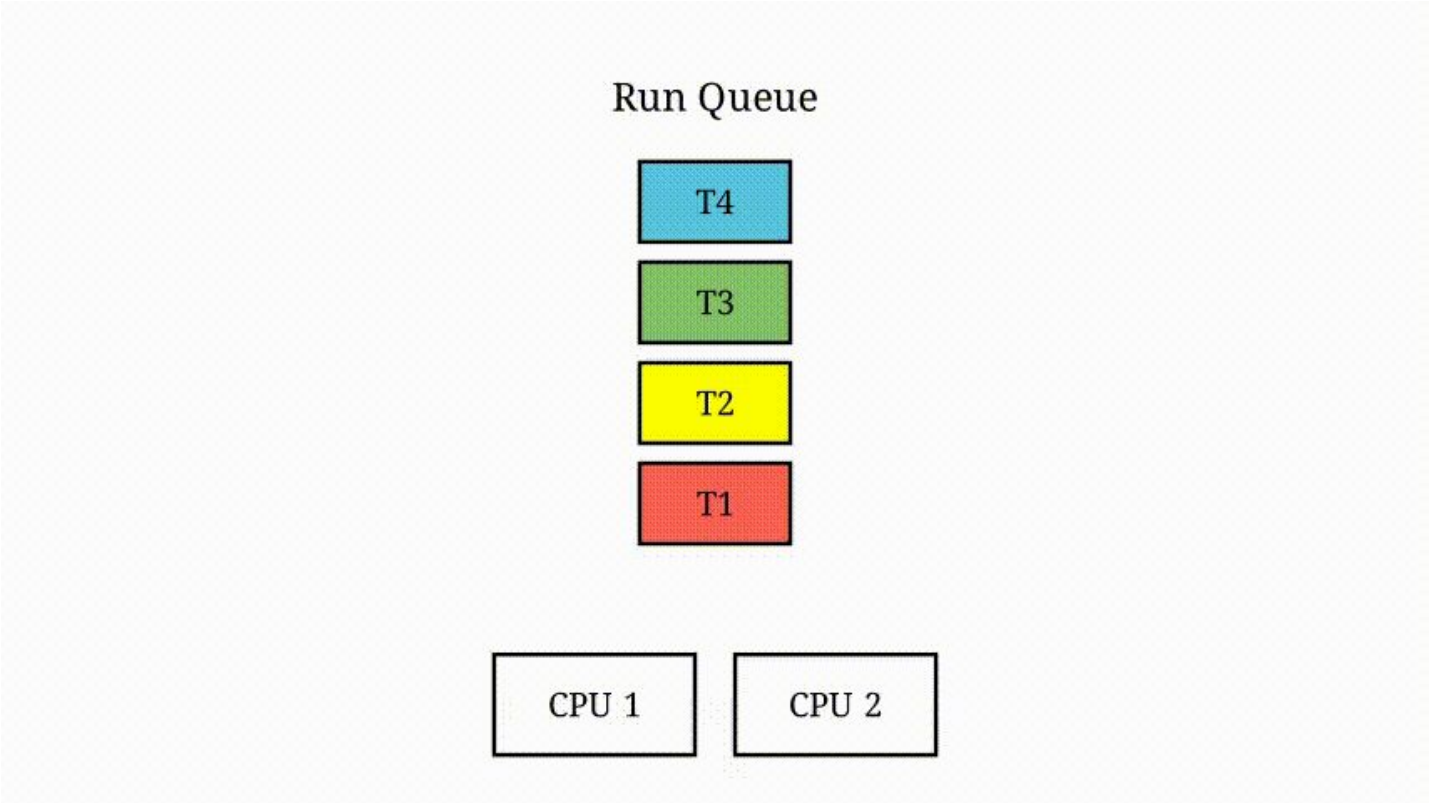
~2.6x throughput

Kernel Recipes

Ingredients

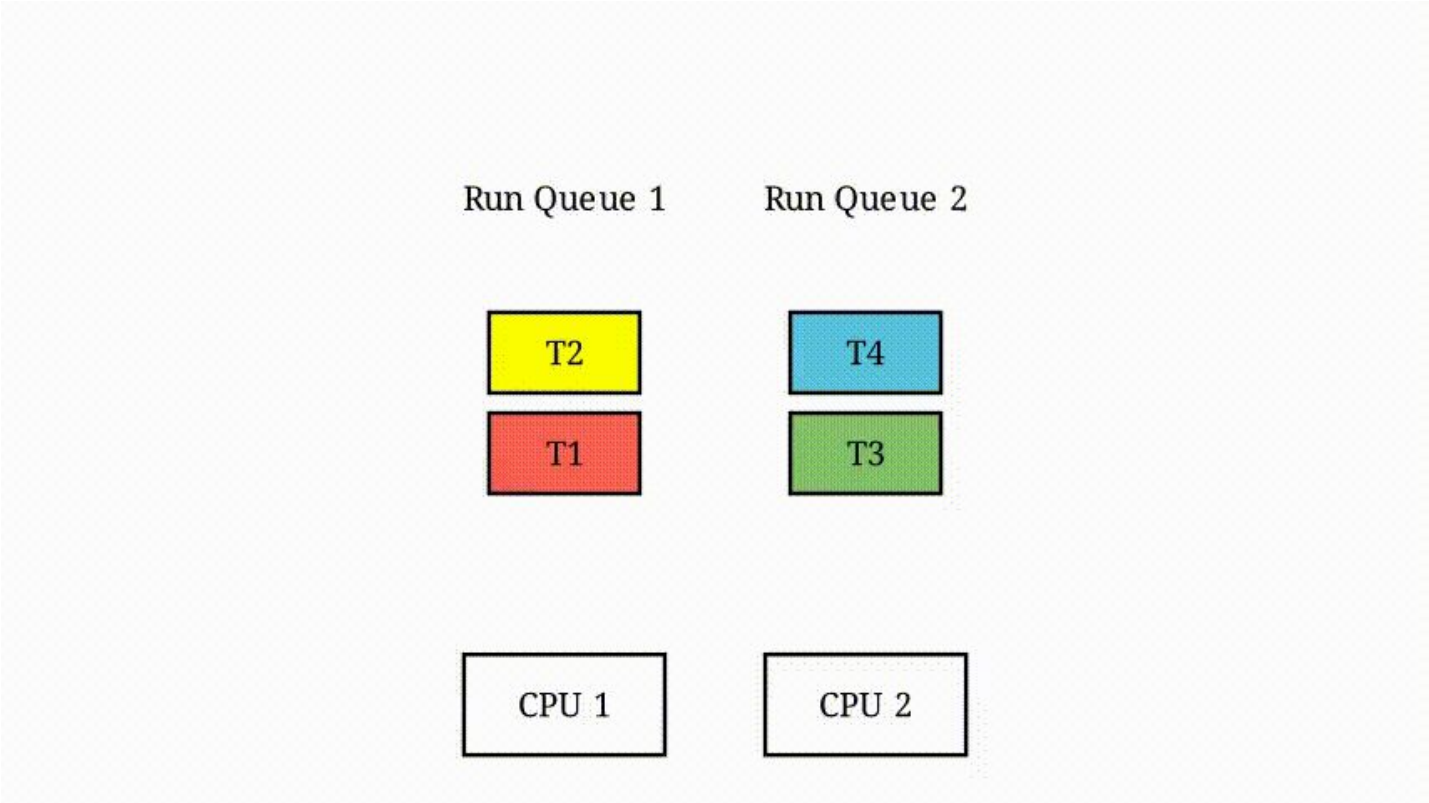


Shared runqueue



Kernel Recipes

Per-CPU runqueues

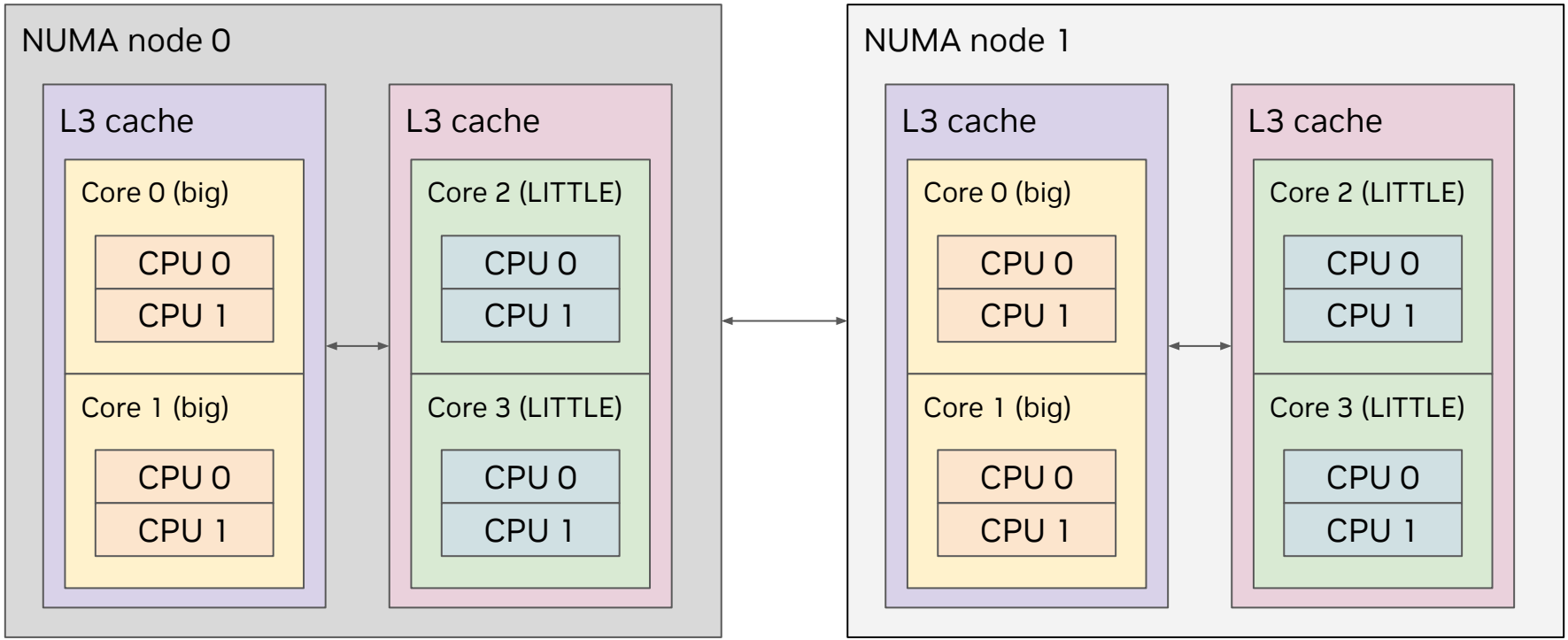


Kernel Recipes

Runtime allocation (fairness)

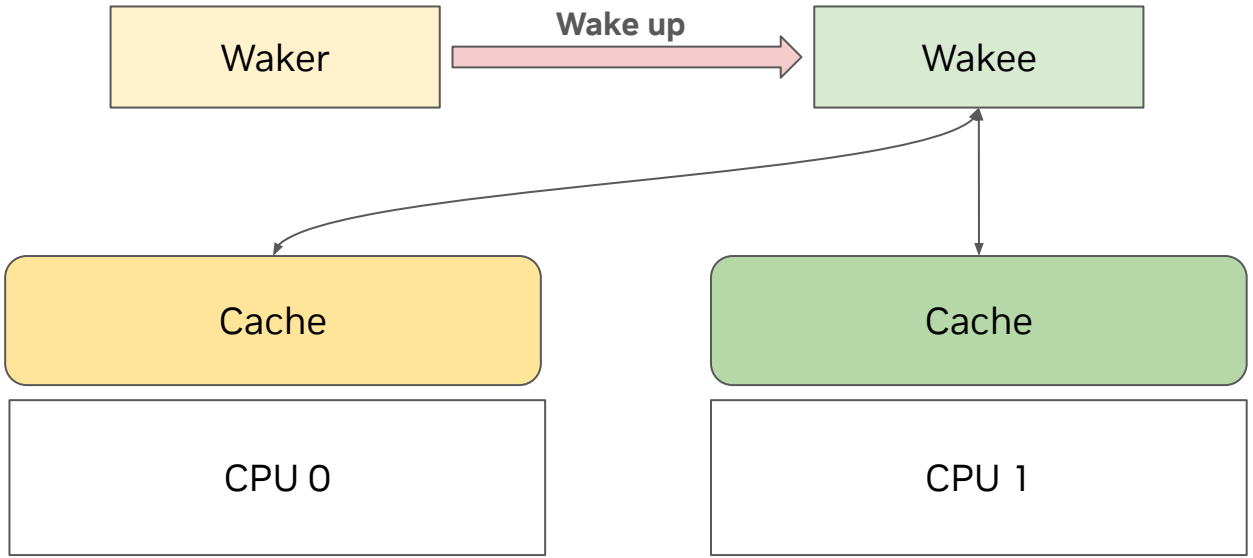
Kernel Recipes

Topology complexity



Kernel Recipes

Task wake-up



Kernel Recipes

Recipes



Recipe #1: The empty scheduler

- Empty scheduler (use sched_ext default)
  - Global run-queue
  - Round-robin scheduler (20ms time slice)
  - Built-in idle CPU selection policy

Kernel Recipes

Recipe #2: The global vs local runqueue scheduler

- Round robin scheduler that can use either a single global queue or multiple per-CPU runqueues
  - Time slice scaled proportionally to the task's priority (weight) and inversely proportional to the amount of contending tasks (fairness)
  - Perfect load balancing (global) vs bad load balancing (local)
  - Not really good if you overload the system (round-robin)
  - Bad for cache locality and scalability (global) vs good for cache locality and scalability (local)

Kernel Recipes

Recipe #3: The “yell at your PC” scheduler

- Adjust the number of CPUs used based on the noise level around your PC
  - Show potential BPF / user space interactions
  - Good for energy consumption (if the environment is quiet)
  - Not usable in public places (i.e., library, open space office, etc.)



Conclusion



Key takeaways

- The one-size-fits-all scheduler approach is no longer sufficient
- CPU allocation can be more effective than time allocation
- Shared queues vs local queues can be relevant
- Prioritizing waker –> wakee pipelines can help improve responsiveness
- Hybrid schedulers (BPF + user space) have great potential

Kernel Recipes

References

- Demo schedulers: https://github.com/sched-ext/scx/tree/kr-demo



Kernel Recipes

NVIDIA

Questions



Kernel Recipes